

Complete Antigravity Prompt Pack

Mini Project 14 - Pharmacovigilance Adverse Event Intake and Triage Service

Order of use

- **Step 0.** Member A runs **Prompt 0 (Pre-phase setup)** once in an empty folder, then pushes to the shared remote. Members B and C clone and check out their own branch.
- **Step 1.** Each member opens Antigravity in the repo and pastes the **Session starter** (change the member letter and branch).
- **Step 2.** Paste your prompt for Phase 1, check the proof it prints, then type "go" for the next phase. Never skip a STOP.
- **Sync points.** After Phase 1 (docker compose up works) and after Phase 3 (reference case correct) everyone merges to main and pulls.
- **Design prompt.** Member A pastes it once /v1/runs and /trace work (after Phase 4). It is the web client (A8).
- **Post-phase prompts.** After Phase 5 (or 6 if you got there): each member pastes their own post-phase prompt. These do integration, freezing, audit and viva preparation.
- **Always:** open the real PDFs and check every clause the system cites for the reference case. Replace <DRUG> in Member C's Phase 1 prompt before pasting.

Where	Prompt	Who
Before everything	Prompt 0: Pre-phase setup	A (once)
Start of each session	Session starter	A, B, C
Phases 1-6	Phase prompts (3 per phase)	A, B, C
After Phase 4	Design prompt (web client)	A
After Phase 5/6	Post-phase prompts	A, B, C
Any time	When something breaks	Whoever hits it

Prompt 0: Pre-phase setup

Member A runs this once, in an empty folder, before anything else.

PRE-PHASE SETUP | run ONCE by Member A in an empty repo folder | do not build features
GOAL: create the shared foundation so three people can work in parallel without conflicts.

1. Check tools: git, docker, docker compose, python3.11. Print versions. If anything is missing, say so and STOP.
 2. git init. Create branches: main, a/platform, b/agents, c/data. Do this step on main.
 3. Create the folder skeleton. Put a one-line README.md in each stating the owner:
A: api/ worker/ stub_llm/ infra/ loadtest/ shared/ scripts/
B: agents/ llm/ prompts/ guardrails/
C: pipeline/ corpus/ eval/ docs/
shared/ is written by A only; B and C treat it as read-only.
 4. Create AGENTS.md at the repo root with the GLOBAL RULES block below, verbatim, plus the folder-ownership table above and the frozen-interface summary from shared/contracts.py.
 5. Create shared/contracts.py (Pydantic v2 models + typing.Protocol), containing:
 Passage(id, text, source_doc, page, clause_id, section, score)
 Citation(passage_id, quote)
 ToolCall(step, tool, input, output_summary, latency_ms, ok)
 TraceStep(step, agent, kind, latency_ms, prompt_tokens, completion_tokens, cost_usd, detail)
 RunResult(run_id, status: completed|escalated|failed,
 routing: expedited|periodic|non_reportable|None, routing_basis,
 seriousness{is_serious, criteria[]}, expectedness{is_listed, matched_term},
 citations[Citation], escalation_reason, prompt_versions{agent: version},
 tool_calls[ToolCall], trace[TraceStep])
 class Retriever(Protocol): search(query, k=5) -> list[Passage];
 get_clause(clause_id) -> Passage | None
 class GraphRunner(Protocol): run_graph(report: dict, run_id: str) -> RunResult
 6. .gitignore: .env, __pycache__, .venv, chroma/index directories, *.log, node_modules.
 .env.example with EVERY variable below, safe defaults, empty secrets:
 LLM_PROVIDER=stub LLM_API_KEY= LLM_MODEL= LLM_BASE_URL= REDIS_URL=
 CHROMA_DIR= EMBED_MODEL= PROMPT_VERSION=v1 OTLP_ENDPOINT= STUB_DELAY_MS=
 STUB_FAIL=false MAX_STEPS= MAX_TOOL_CALLS= RUN_TIMEOUT_S= RATE_LIMIT_PER_MIN=
 PRICE_PER_1K_PROMPT= PRICE_PER_1K_COMPLETION= BREAKER_FAILS= BREAKER_RESET_S=
 7. Makefile targets: up, down, logs, test, ingest, eval, load, scan-secrets, clean-clone-test (targets may just echo "not implemented yet" for now).
 8. scripts/scan_secrets.sh: scan the working tree AND git history for key-like strings (AIza..., sk-..., api_key=, bearer tokens); exit non-zero on a hit. scripts/install_hooks.sh installs it as a git pre-commit hook. Run the install script.
 9. README.md skeleton with headings only: Overview, Quick start, Architecture, Services, Environment variables, Ingest, API, Evaluation, Load test, Runbook, Troubleshooting.
 10. Commit on main: "chore: project foundation". Print `git log --oneline`, the tree, and the exact commands teammates must run to clone and switch to their branches.
- STOP.

GLOBAL RULES (paste verbatim into AGENTS.md)

- Python 3.11. Docker Compose is the only way the system runs. Default to the STUB LLM.
- Never invent regulatory clauses, deadlines or package-insert text. The system may cite only text retrieved from ./corpus. When writing tests or docs, read the real PDFs and quote them.
- Synthetic data only. Never real patient data.
- No credentials, endpoints or model identifiers in the repo. All config via environment.
- The system only RECOMMENDS. It never submits a regulatory report. A qualified person decides.
- An unverified or incomplete run must never be returned as a confident answer.
- Edit only the folders you own. If you need a change elsewhere, tell the owner in your report.
- Frozen interfaces live in shared/contracts.py. Do not change them without team agreement.
- Work in small steps. Run what you build. Show proof (commands + output). Commit to your branch. Then STOP and wait for "go".

Session starter (every member, every new session)

Read AGENTS.md and shared/contracts.py completely. I am Member <A|B|C>, working on branch <a/platform | b/agents | c/data>.
Confirm in 5 lines: (1) the folders I may edit, (2) the frozen interfaces, (3) the global rules, (4) which phase we are on, (5) what you will NOT do without my "go".
Do not start building. Wait for my phase prompt.

Phase 1: Skeleton, corpus and LLM interface

Sync point: everyone merges to main when done. docker compose up must work from a clean clone.

Phase 1 - Member A - Platform

PHASE 1 | MEMBER A (Platform) | branch a/platform
GOAL: docker compose up serves a valid request end to end.
READ FIRST: AGENTS.md, shared/contracts.py.
TASKS

1. docker-compose.yml with services api, worker, redis, stub-llm. Healthcheck on each; depends_on with condition: service_healthy. Worker must be scalable (no container_name, no fixed host port).
2. api/models.py: Pydantic v2 report model. Validate types; ranges (patient_age 0-120); enums (patient_sex, reporter_type, outcome); cross-field (event_date >= drug_start_date, no future dates); reaction_description required, max 2000 chars. Export api/contract/report.schema.json and write api/contract/README.md (field table, rules, one valid and one invalid example).
3. POST /v1/reports validates FIRST. Invalid -> store in quarantine (SQLite table or Redis list) with field-level errors, return 422, enqueue NOTHING. Valid -> enqueue an RQ job, return 202 {run_id} immediately.
4. GET /v1/runs/{run_id} -> status + result. GET /health. GET /metrics (Prometheus).
5. worker/: RQ worker calling run_graph. Until Member B merges, use worker/placeholder.py returning a fixed RunResult (swap by changing one import).
6. stub_llm/: FastAPI returning fixed JSON after STUB_DELAY_MS; HTTP 500 when STUB_FAIL=true.

PROOF TO SHOW

- docker compose up with all services healthy.
- curl valid report: 202, then GET shows completed.
- curl invalid report: 422 with field errors; quarantine record present; Redis queue length unchanged (proves no agent run started).

DO NOT edit agents/, pipeline/, prompts/. Commit to a/platform. STOP.

Phase 1 - Member B - Agent Core

PHASE 1 | MEMBER B (Agent Core) | branch b/agents
GOAL: LLM abstraction plus a runnable agent skeleton that returns a valid RunResult.
READ FIRST: AGENTS.md, shared/contracts.py.
TASKS

1. llm/client.py: LLMClient.generate(prompt, system, **kw) -> {text, prompt_tokens, completion_tokens, latency_ms, model}. Providers: "stub" (HTTP call to LLM_BASE_URL) and "gemini" (key and model from env). Chosen ONLY by LLM_PROVIDER. Nothing hardcoded.
2. Every model call takes a timeout (default from env).
3. prompts/ as versioned files: prompts/assessor/v1.md, prompts/verifier/v1.md and a loader returning (text, version). PROMPT_VERSION selects the version.
4. agents/: LangGraph graph with nodes assessor -> verifier. State holds run_id, report, retrieved passages, tool_log, step counter, prompt versions.
5. agents/tools.py: signatures for search_corpus, get_clause, get_request_fields, rewrite_query, backed by a MockRetriever returning hard-coded Passage objects.
6. agents/run.py: run_graph(report, run_id) -> RunResult (matches shared/contracts.py).
7. Unit tests using the stub provider.

PROOF TO SHOW

- pytest output; run_graph on a sample report printing a valid RunResult (model_validate passes).

DO NOT edit api/, worker/, pipeline/. Commit to b/agents. STOP.

Phase 1 - Member C - Data, RAG and Eval

PHASE 1 | MEMBER C (Data, RAG, Eval) | branch c/data

GOAL: lock the corpus and the reference case with verbatim evidence.

READ FIRST: AGENTS.md, shared/contracts.py.

INPUT: ./corpus contains NDCT Rules 2019 (adverse event provisions), the IPC pharmacovigilance guidance for MAHs, and the package insert for <DRUG>.

TASKS

1. Confirm each PDF has a text layer. If scanned, OCR it and note that in docs/corpus.md.
2. docs/corpus.md: for each document give title, version/date, source URL, page count.
3. Find the EXACT passages the reference case depends on and quote each verbatim with document, page and clause/section id:
 - (a) the insert's listed severe skin reactions,
 - (b) the seriousness criteria (hospitalisation),
 - (c) the reporting provision for expedited vs periodic,
 - (d) any provision that modifies that timeline.
4. docs/reference_case.md: the case text adapted to <DRUG>; the lay-to-medical mapping ("skin peeling" -> terms the insert actually uses); expected seriousness, expectedness and routing; supporting passages; the synthetic test values (age, sex, dates, outcome).
5. pipeline/models.py: Passage dataclass matching shared/contracts.py.

PROOF TO SHOW

- docs/corpus.md and docs/reference_case.md with verbatim quotes I can present at Evaluation 1.
 - An explicit list of anything you could NOT find in the documents. Do not fill gaps from memory.
- DO NOT edit agents/, api/. Commit to c/data. STOP.

Phase 2: Ingestion pipeline and retrieval

Sync point: Member B swaps MockRetriever for Member C's real retriever (one env change).

Phase 2 - Member A - Platform

PHASE 2 | MEMBER A (Platform)

GOAL: platform ready for the real index, monitored, and reproducible from a clean clone.

TASKS

1. Add prometheus (scrapes api and worker) and jaeger to docker-compose. Add infra/prometheus.yml.
2. Add a one-shot "ingest" service running `python -m pipeline ingest` with shared volumes for corpus and CHROMA_DIR. Worker depends on it with condition: service_completed_successfully.
3. Confirm the worker container can import Member C's retriever (agree the import path; do not edit pipeline/).
4. /health reports dependency status: redis, index present (index_meta.json), stub-llm reachable.
5. Clean-clone test: fresh clone in a temp folder, cp .env.example .env, docker compose up, valid request completes. Append any REAL failure to docs/incident_log.md (ask C for format).

PROOF TO SHOW

- Prometheus targets page (or API output) showing api and worker UP.
- Clean-clone run log with a completed request.

DO NOT edit pipeline/, agents/. STOP.

Phase 2 - Member B - Agent Core

PHASE 2 | MEMBER B (Agent Core)

GOAL: tools are ready for real retrieval and the lay-to-medical rewrite works.

TASKS

1. Refactor agents/tools.py so retrieval sits behind the Retriever protocol from shared/contracts.py. Provide MockRetriever and RealRetrieverAdapter (imports Member C's retrieval). Choose via env RETRIEVER=mock|real.
2. rewrite_query(lay_text): LLM tool returning 3-5 candidate medical terms. Prompt in prompts/rewrite/v1.md. Include a deterministic fallback dictionary for stub/offline use.
3. get_request_fields(run_id): returns the original report fields for the run.
4. Tool wrapper recording EVERY call (tool, input, output summary, latency, ok) into state.tool_log, enforcing MAX_TOOL_CALLS and a per-call timeout.
5. Tests for each tool using MockRetriever.

PROOF TO SHOW

- pytest output; a printed tool_log for a sample run; RETRIEVER=real works once C's index exists.

DO NOT edit pipeline/, api/. STOP.

Phase 2 - Member C - Data, RAG and Eval

PHASE 2 | MEMBER C (Data, RAG, Eval)

GOAL: an idempotent corpus pipeline and retrieval that meets the frozen interface.

TASKS

1. CLI: python -m pipeline ingest [--reembed <doc>] [--force]. Steps: read -> parse (page-aware; keep clause numbers and section headings) -> chunk -> embed locally with sentence-transformers (model name from EMBED_MODEL) -> index in Chroma at CHROMA_DIR.
2. Idempotent: content hash per chunk; unchanged chunks skipped; --reembed re-embeds one document without rebuilding the whole index.
3. Chunking that addresses the vocabulary gap: clause-boundary chunks, heading prepended to each chunk, and for the package insert an extra set of chunks holding the listed adverse-reaction terms. Document this in docs/chunking.md, explaining how it handles lay-vs-medical wording.
4. pipeline/retrieval.py implementing Retriever: search(query, k) and get_clause(clause_id).
5. Freshness: write index_meta.json (build time, source doc versions, chunk count, chunks/sec) and expose the same values as Prometheus gauges.
6. Test with the lay phrase "skin started peeling badly" and with the medical term. Show the top-5 for each with scores.

PROOF TO SHOW

- ingest run twice (second run skips unchanged chunks); index_meta.json; top-5 tables; a clear statement of whether the passages needed for the reference case are retrieved.

DO NOT edit agents/, api/. STOP.

Phase 3: Agents (the core of the project)

Sync point: B and C sit together. Checkpoint 2: reference case correct end to end; out-of-corpus drug escalates.

Phase 3 - Member A - Platform

PHASE 3 | MEMBER A (Platform)

GOAL: the real graph runs in the worker; failures are handled honestly.

TASKS

1. Replace worker/placeholder.py with agents.run.run_graph. Any worker exception -> run stored as status "failed" with reason. Never a confident answer for an incomplete run.
2. Run statuses queued/running/completed/escalated/failed with timestamps in GET /v1/runs/{id}.
3. RQ job timeout from RUN_TIMEOUT_S; failed and timed-out jobs land in a failed-jobs path and are visible via the API. Document in api/README why async plus a queue (runs take tens of seconds).
4. Structured JSON logging (api/logging_setup.py) with correlation_id = run_id from API to job to worker.
5. Run the reference case through the full stack; save the JSON to docs/demo/reference_case_result.json.

PROOF TO SHOW

- Reference case result via the API; a killed job showing status "failed"; logs sharing one correlation_id.

DO NOT edit agents/. STOP.

Phase 3 - Member B - Agent Core

PHASE 3 | MEMBER B (Agent Core)

GOAL: two real agents with distinct duties and a verified, honest output.

TASKS

1. ASSESSOR agent. Tools: search_corpus, get_clause, get_request_fields, rewrite_query. Retrieval is a tool the agent CHOOSES to call, not a fixed stage. Required path: initial retrieval -> identify the provision that seems to apply -> ask whether another provision modifies it and search for it -> re-query using terms taken from what it read -> draft seriousness, expectedness, routing with citations (passage_id + short quote).
2. VERIFIER agent (separate node, separate prompt, separate view of state). Checks (a) every cited passage_id was retrieved in this run, (b) each quote appears in that passage, (c) the passages support each claim. On failure: back to the Assessor with a DIFFERENT search strategy (max 2 retries), else escalate. An unverified answer is never returned.
3. Escalate with reason when: the reaction cannot be matched to the insert with confidence; timing is unclear (missing or inconsistent dates); the drug is outside the corpus.
4. Bounds MAX_STEPS, MAX_TOOL_CALLS, RUN_TIMEOUT_S; exceeding any -> escalate.
5. Failure behaviour: Verifier crashes -> "escalated". Assessor crashes -> "failed". Document topology (mermaid) plus a failure table in agents/TOPOLOGY.md.
6. Record prompt version for every agent in RunResult.prompt_versions.

PROOF TO SHOW (three outputs)

- (i) reference case correct with citations that resolve;
- (ii) an out-of-corpus drug escalates;
- (iii) a deliberately forced unsupported citation is caught by the Verifier, followed by another search or an escalation.

DO NOT edit pipeline/, api/. STOP.

Phase 3 - Member C - Data, RAG and Eval

PHASE 3 | MEMBER C (Data, RAG, Eval)

GOAL: retrieval good enough for the reference case; first half of the eval set.

TASKS

1. Work beside Member B: run the reference case and inspect what was retrieved at each tool call. Where the needed passage is missed, fix it in YOUR area only: chunking, heading metadata, listed-terms chunks, hybrid (keyword + vector) search, or a synonym table pipeline/synonyms.json. Do not touch agents/.
2. docs/retrieval_tuning.md: recall@5 of the expected passage for the reference case plus 5 other queries, before and after fix.
3. Start eval/scenarios.jsonl with 10 scenarios. Fields: id, description, report (synthetic), expected_action (answer/escalate), expected_routing, expected_seriousness, expected_expectedness, supporting_passage_id, supporting_quote (verbatim from corpus). Include: reference case, drug outside corpus, missing dates, listed non-serious, unlisted serious, one prompt-injection attempt.

PROOF TO SHOW

- The tuning table with measured improvement; 10 scenarios; a check that each quote exists in the corpus text.

DO NOT edit agents/, api/. STOP.

Phase 4: Safety, resilience and observability

Sync point: A and B agree how breaker state and step logs appear in the trace.

Phase 4 - Member A - Platform

PHASE 4 | MEMBER A (Platform)

GOAL: any run can be reconstructed from its run_id.

TASKS

1. OpenTelemetry in api and worker, exported to Jaeger via OTLP_ENDPOINT. One trace per run; child spans per agent step and tool call, built from state.tool_log and trace via a hook in the worker (do not edit agents/).
2. Prometheus metrics: request latency histogram, end-to-end run latency (submit -> final), error rate, throughput, queue depth, runs by status, escalations.
3. GET /v1/runs/{run_id}/trace returns every tool call, query, retrieved passage ids, prompt versions, per-step latency, tokens and cost, verification outcome. Persist traces (SQLite or Redis) so they survive worker restarts.
4. correlation_id (= run_id) appears in api, worker and every agent-step log line.

PROOF TO SHOW

- One run: Jaeger trace screenshot or listing, /trace JSON, and logs filtered by run_id.

DO NOT edit agents/. STOP.

Phase 4 - Member B - Agent Core

PHASE 4 | MEMBER B (Agent Core)

GOAL: guardrails and resilience that fail safe.

TASKS

1. guardrails/injection.py: check EVERY free-text field before it reaches a model (reaction_description, narrative, ...): pattern/heuristic detection for instruction override, role hijack, tool-call smuggling; wrap user text in delimiters with a system instruction to treat it as data. Detected -> block, log, escalate with reason "possible prompt injection".
2. guardrails/output.py: block any output that states a final regulatory submission or decision ("submitted", "approved for filing", "I have reported"). Outputs must read as recommendations for a qualified person.
3. guardrails/toxicity.py: simple toxicity check on input and output; document its limits.
4. llm/resilience.py: timeout on every model call; bounded retry with exponential backoff and jitter; circuit breaker (closed/open/half-open, thresholds BREAKER_FAILS, BREAKER_RESET_S); fallback: breaker open -> run ends "escalated" with reason "LLM unavailable". Expose breaker state as a Prometheus gauge and inside the trace.
5. Record prompt tokens, completion tokens and cost (PRICE_PER_1K_* from env) per step.

PROOF TO SHOW

- STUB_FAIL=true: breaker opens, runs escalate, breaker recovers when the stub is healthy. Logs.

- An injection attempt blocked, with the guardrail response.

- An output-filter test that blocks a "submitted" phrasing.

DO NOT edit api/, pipeline/. STOP.

Phase 4 - Member C - Data, RAG and Eval

PHASE 4 | MEMBER C (Data, RAG, Eval)

GOAL: complete the 20-scenario eval set; start runbook and incident log.

TASKS

1. Extend eval/scenarios.jsonl to 20 scenarios covering: reference case; listed vs unlisted; serious vs non-serious; expedited vs periodic vs non_reportable; missing or inconsistent dates; unknown drug; at least 3 lay phrasings of the skin reaction; 2 injection attempts; 1 toxic input; 1 report where a modifying provision changes the routing.
2. eval/validate_set.py: verifies every supporting quote exists in the corpus text and every scenario report passes the data contract.
3. docs/incident_log.md with template columns: date, symptom, detection, root cause, fix, prevention. Add REAL failures met so far (ask A and B). Never invent entries.
4. docs/runbook.md draft for the two most likely failures: (1) LLM provider outage / breaker open, (2) retrieval index missing or stale. Each with symptom, confirmation commands, recovery.

PROOF TO SHOW

- validate_set.py output (20 valid); runbook draft; incident log with real entries only.

DO NOT edit agents/, api/. STOP.

Phase 5: Evaluation and load testing

Sync point: Member A must COMMIT docs/SLO.md before any Locust run.

Phase 5 - Member A - Platform

PHASE 5 | MEMBER A (Platform)

GOAL: declared SLO, honest load results, scaling experiment.

TASKS

1. FIRST write docs/SLO.md: the p95 target for a COMPLETE run (submission to final result), justified against section 1 of the brief (expedited reports have fixed deadlines, so triage latency is small against the deadline, but a qualified person needs results promptly). State the number, stub delay and assumptions. COMMIT IT. Only then continue.
2. loadtest/locustfile.py: POST a valid report, poll /v1/runs/{id} until terminal, and record submission-to-final-result time as a custom Locust metric (not just the submit call). LLM_PROVIDER=stub with a realistic STUB_DELAY_MS.
3. Run at rising concurrency with 1 worker. Report RPS, p50, p95 against the SLO, state MET or NOT MET, and the concurrency where it fails plus the responsible component.
4. Repeat with docker compose up --scale worker=3. Report throughput and p95 change. If scaling did not help, identify the true bottleneck with evidence (CPU, Redis, retrieval, embedding).
5. Write docs/load_results.md with tables. Never edit the target afterwards.

PROOF TO SHOW

- The SLO commit hash (earlier than any load result), results tables, honest conclusion.

DO NOT edit agents/, pipeline/. STOP.

Phase 5 - Member B - Agent Core

PHASE 5 | MEMBER B (Agent Core)

GOAL: two prompt variants, failure fixes, and one flagged deliberate change.

TASKS

1. prompts/assessor/v2.md: a deliberately different variant (for example stricter citation format or more explicit search steps). PROMPT_VERSION switches v1/v2; the version is recorded in every result.
2. Make the batch evaluation runnable against either version with no code edits.
3. When Member C's eval reports failures, diagnose each: retrieval miss (tell C); prompt problem (fix in prompts/); verifier too lax or strict (agents/); guardrail false positive (guardrails/). Keep agents/CHANGES.md: failure, cause, change, effect.
4. Pick ONE deliberate change for the before/after test (for example the modifying-provision search step, or a stricter verifier) behind an env flag so both states can run.

PROOF TO SHOW

- v1 and v2 outputs on the same scenario; flag off vs on behaviour; CHANGES.md.

DO NOT edit api/, pipeline/. STOP.

Phase 5 - Member C - Data, RAG and Eval

PHASE 5 | MEMBER C (Data, RAG, Eval)

GOAL: evaluation harness with four metrics and a before/after result.

TASKS

1. CLI: python -m eval.run --prompt-version v1 [--set eval/scenarios.jsonl] [--direct]. Runs all scenarios through API -> queue -> worker, or directly via run_graph with --direct. Use TruLens or a documented equivalent.
2. docs/metrics.md, stating exactly how each is computed:
 - Groundedness: share of answer claims supported by retrieved passages.
 - Context relevance: relevance of retrieved passages to the query, scored 0-1.
 - Faithfulness: agreement between the answer and the passages it cites.
 - Hallucination rate: share of runs with a citation that does not resolve to retrieved text or a claim the passages do not support.Also report routing accuracy and correct-escalation rate.
3. Compare prompt v1 vs v2 on the same set: which is better, on which metric, by how much.
4. Run Member B's deliberate change: report all four metrics BEFORE and AFTER in docs/eval_results.md with per-scenario failures.
5. Note which LLM (stub or real) produced each result; use the real model sparingly.

PROOF TO SHOW

- eval_results.md tables; the v1/v2 comparison; the before/after table.

DO NOT edit agents/, api/. STOP.

Phase 6: Optional extras and polish (only if time remains)

Do this only after Phases 1-5 work. Whoever finishes early takes the next item.

Phase 6 - Member A - Platform

PHASE 6 | MEMBER A (Platform)

GOAL: cost controls, caching, dashboard, feedback.

TASKS

1. Per-client rate limit on POST /v1/reports (Redis token bucket, client id from a header, RATE_LIMIT_PER_MIN). Return 429 with Retry-After. Document why it is a cost control for a multi-step agent.
2. Redis cache of whole-run results keyed by a hash of the normalised report. Measure the effect on p95 and cost per run with Locust using a repeat-request mix; add to docs/load_results.md.
3. Grafana with a provisioned dashboard: p50/p95 latency, error rate, throughput, availability, CPU and memory per container (cAdvisor), breaker state, queue depth.
4. POST /v1/runs/{run_id}/feedback (satisfactory bool + reason) persisted; GET /v1/feedback/summary.
5. (The web client is the separate DESIGN prompt.)

PROOF TO SHOW

- 429 demo; before/after cache numbers; dashboard screenshot; feedback summary output.

DO NOT edit agents/, pipeline/. STOP.

Phase 6 - Member B - Agent Core

PHASE 6 | MEMBER B (Agent Core)

GOAL: cost report and live quality sampling.

TASKS

1. Cost per 100 runs from recorded token usage; identify the largest single cost driver (which agent step or tool). Write agents/COST.md.
2. Live sampling: a background job scoring about 10% of completed runs with Member C's scorers (import them), storing timestamped scores so quality over time can be charted.
3. Review guardrails for false positives on legitimate clinical narratives (words like "ignore" or "system"); tune and record what changed.

PROOF TO SHOW

- COST.md with numbers; a table of live-sample scores with timestamps.

DO NOT edit api/, pipeline/. STOP.

Phase 6 - Member C - Data, RAG and Eval

PHASE 6 | MEMBER C (Data, RAG, Eval)

GOAL: documentation a stranger can follow, plus the report skeleton.

TASKS

1. Finish docs/runbook.md and docs/incident_log.md (real entries only).
2. README.md: prerequisites, clean-clone steps (git clone, cp .env.example .env, docker compose up), ingest, submitting a request with curl, running eval, running Locust. TEST it in a fresh folder exactly as written; fix every gap.
3. docs/report.md skeleton: problem and scope; agent design (topology, tools, verification, loop limits); architecture diagram with deployment boundaries marked; design decisions and rejected alternatives; before/after eval results; SLO, measured p95, bottleneck, cost per 100 runs; what did not work. Pull numbers from eval_results.md, load_results.md, agents/COST.md.
4. docs/demo_script.md: the 10 demonstration items, each with the exact command or click.

PROOF TO SHOW

- Clean-folder README test log; the report skeleton with real numbers filled in where they exist.

DO NOT edit agents/, api/. STOP.

Design prompt: premium light-theme web client

Member A, after Phase 4. Produces the A8 client with a professional look.

DESIGN PROMPT | MEMBER A (Platform) | folder api/web/ | run after Phase 4
GOAL: a premium, professional, LIGHT-theme single-page web client served by the API at "/".
Requirement A8 only asks for function, but this is what a reviewer sees first. Make it look like a modern clinical-grade SaaS product: calm, spacious, trustworthy.

TECH CONSTRAINTS

- Plain HTML + CSS + JS in api/web/ (index.html, styles.css, app.js). No frameworks, no CDN, no external fonts or images: it must work offline inside Docker. Inline SVG for icons.
- Talks only to this API: POST /v1/reports, GET /v1/runs/{id}, GET /v1/runs/{id}/trace, POST /v1/runs/{id}/feedback, GET /health. Render ONLY fields the API returns. Never invent data or citations.

DESIGN TOKENS (CSS variables)

- Background #F6F8FB; surface #FFFFFF; border #E3E8EF; ink #0F172A; muted #5B6B7F.
- Brand navy #12355B; accent teal #0E8F86; focus ring #2F80ED.
- Status: expedited #C2410C on #FFF1E8; periodic #1D4ED8 on #EAF1FF; non-reportable #475569 on #EEF2F6; escalated #B45309 on #FFF7E0; verified #15803D on #E8F7EE; error #B42318 on #FEECEB.
- Font stack: -apple-system, "Segoe UI", Inter, Roboto, Helvetica, Arial, sans-serif.
Scale 12/14/16/20/28 px; weights 400, 500, 600; monospace for run ids and quotes.
- 8px spacing grid; radius 12px cards, 8px inputs; soft shadow 0 1px 2px rgba(15,23,42,.06), 0 8px 24px rgba(15,23,42,.06); 150-200ms ease transitions; honour prefers-reduced-motion.

LAYOUT

- Top bar: logo mark (inline SVG) + "PV Triage", subtitle "Adverse event intake and assessment", a health dot from /health, an environment chip.
- Desktop (>=1024px): 2 columns. Left, sticky card "New report" (form). Right, "Assessment" panel.
Tablet/mobile: stacked. Max content width 1200px. Test at 1440px and 390px.

COMPONENTS AND STATES

1. Form: sections "Patient", "Suspect drug", "Event". Labels above inputs, helper text, inline field errors mapped from the 422 field-level errors. "Load reference case" button fills the documented synthetic case. Submit button shows a spinner and disables while running.
2. Empty state: quiet illustration (inline SVG), one sentence, the reference-case shortcut.
3. Running state: horizontal stepper (Queued, Retrieving, Assessing, Verifying, Done) driven by polling every 1.5 s, elapsed-time counter, aria-live announcements. Timeout message if the run exceeds the client limit; do not show a result for a failed run.
4. Result card: header with routing badge, run id (copy button), prompt version chips, end-to-end latency. Three summary tiles: Seriousness, Expectedness, Routing, each with its basis text. "Verified" badge showing verification outcome.
5. Citations: list of cards, each with document name, page, clause id, the quoted text with the quote highlighted, and an expander for the full retrieved passage.
6. Escalation state (amber, never green): reason, what could not be established, and a "Recommended human action" line. Clearly distinct from a completed result.
7. Trace drawer: timeline of agent steps and tool calls with durations, tokens and cost from /trace. Collapsible; monospace details.
8. Feedback: thumbs up / thumbs down with a reason text box; confirmation toast.
9. Persistent footer note: "Decision support only. A qualified person for pharmacovigilance makes every determination. This tool never submits a regulatory report."
10. Error states for network failure, 429 (show Retry-After), 422 and failed runs.

QUALITY BAR

- WCAG AA contrast, visible focus rings, full keyboard operation, labelled form controls, aria-live for status, semantic headings. No layout shift when results appear.
- Microcopy is plain and precise; no exclamation marks; no emoji.

PROOF TO SHOW

- Screenshots at 1440px and 390px for: empty, running, completed, escalated, validation error. Save them in docs/screens/ (use a headless browser if available; otherwise describe how to capture them). Confirm the page works with the browser offline after load.

DO NOT edit agents/, pipeline/. Commit to a/platform. STOP.

Post-phase prompts (after Phase 5 or 6)

Post-phase - Member A - Platform

POST-PHASE | MEMBER A (Platform) | integration, freeze and demo automation

GOAL: a system that starts from a clean clone and a scripted demo that never surprises you.

TASKS

1. Merge b/agents and c/data into main. Resolve conflicts without deleting others' work. Run the full test suite and fix breakages caused by the merge.
2. make clean-clone-test: script that clones the repo into /tmp, copies .env.example to .env, runs docker compose up -d --build, waits for all healthchecks, submits the reference case, asserts a valid terminal result, then tears everything down. Run it. Fix every failure. Record each REAL failure in docs/incident_log.md.
3. scripts/demo.sh: the 10 demonstration items as numbered steps with a pause between each (valid run with citations; traced run; malformed request + quarantine + queue unchanged; out-of-corpus escalation; verifier catch; injection attempt; breaker open + degradation via STUB_FAIL; Locust summary; --scale comparison; reference case). Each step prints what it shows.
4. Security: run scripts/scan_secrets.sh on tree and history; confirm .env is not tracked; grep the repo for hardcoded model names, endpoints and keys. Report results.
5. Startup timing from a clean clone (with and without a pre-built index); write the expected duration in the README.
6. Tag v1.0 only if everything passes. Print a pass/fail checklist of the 10 demo items.

STOP.

Post-phase - Member B - Agent Core

POST-PHASE | MEMBER B (Agent Core) | robustness and viva preparation

GOAL: prove the agents fail safe, and be able to explain every design choice.

TASKS

1. Soak test: 50 runs on the stub with mixed scenarios from eval/scenarios.jsonl. Report the completed/escalated/failed counts and any run that returned a confident answer without passing verification (must be zero).
2. Chaos tests, each with logs: kill a worker mid-run; restart Redis mid-run; STUB_FAIL=true mid-run; exceed MAX_TOOL_CALLS; exceed RUN_TIMEOUT_S. Expected outcome each time: status failed or escalated, never completed. Fix anything that differs.
3. Confirm demo items 5-8 behave exactly as scripted in scripts/demo.sh (out-of-corpus escalation, verifier catch, injection block, breaker open and recovery).
4. Confirm prompt_versions is present in every RunResult and every trace.
5. Write agents/VIVA.md: 15 likely examiner questions with short answers grounded in this code (why two agents and not two prompts; what the verifier checks; what happens when a tool fails; how loop bounds work; breaker states; why an unverified answer is never returned; what was rejected and why).

PROOF TO SHOW

- Soak and chaos summaries, VIVA.md. Commit to b/agents. STOP.

Post-phase - Member C - Data, RAG and Eval

POST-PHASE | MEMBER C (Data, RAG, Eval) | final numbers, report and documentation audit

GOAL: every deliverable exists, is correct, and every number comes from a result file.

TASKS

1. Final eval run of all 20 scenarios with the final configuration; update docs/eval_results.md (four metrics, v1 vs v2, before/after). Record which LLM produced each number.
2. Re-verify the reference case against the REAL PDFs: every cited clause and quote must exist in the corpus text at the stated page. Write docs/reference_verification.md with the check.
3. Finish docs/report.md: fill every number from docs/eval_results.md, docs/load_results.md and agents/COST.md; include the architecture diagram with deployment boundaries marked, the agent topology, rejected alternatives, and a candid "what did not work" section. Export it to PDF (docs/report.pdf).
4. Deliverables audit script scripts/audit_deliverables.py: checks that README, runbook, incident log (>=1 real entry), .env.example, eval set (>=20 scenarios), SLO.md (committed before the load results), report, and demo script all exist. Run it; fix gaps.
5. README test as a stranger in a fresh folder, following only the README. Fix every gap.
6. docs/viva_questions.md: 15 likely questions on data, retrieval, evaluation, SLO and bottleneck, with short answers taken from the results.

PROOF TO SHOW

- Audit script output (all pass), README test log, report PDF path. Commit to c/data. STOP.

When something breaks (any member, any time)

WHEN SOMETHING BREAKS (use in any session)

Stop adding features. Reproduce the failure and show the exact command and output.

Find the ROOT CAUSE (state it in one sentence, with evidence from logs or code).

Apply the smallest fix inside the folders I own; if the cause is in someone else's folder, write a 5-line note for the owner and stop.

Add a test or check that would have caught it. Re-run the failing step and show it passing.

Append a REAL entry to docs/incident_log.md: date, symptom, detection, root cause, fix, prevention. Do not invent or embellish entries.